ElectronBot v2 - Roadmap (Webcam-follow, Zephyr, PC Software)

> Technical revision based on direct reading of the code (`3.Software/SDK`, `_Tools/AHK-ExpansionPack`). Planning document — no code was executed or modified.

⚠️ Factual corrections (derived from the code)

#	Previous assumption	Reality in the code
C1	"You can move the servo without sending an image"	**False.** The only send path is `Sync()`, which **always uploads the entire frame (172,800 B)** (`SyncTask`: 4×(84×512 + 224-byte tail)). The 6 joints travel in the **32 bytes of the tail**. ⇒ control loop rate = full-frame upload rate; the display is always being fed.
C2	Reading angles is straightforward	`GetJointAngles` only reflects the **last** `Sync`, and the wrapper reads **2×** with a `Sleep 10` in between. "Physical-priority" mode costs one cycle + handshake.
C3	Generic limits	**Confirmed:** raw `SetJointAngles(j1..j6)` → **j1=head (±15°)**, j2=left-arm-open, j3=right-arm-raise, j4=right-arm-open, j5=left-arm-raise, **j6=waist/rotation (±90°)**. The webcam rotates **j6**.
C4	"Mirrored camera"	The bridge applies `flip(img, -1)` = **180° rotation** (camera mounted upside down). Detection and the control signal have to undo this.
C5	—	`USB_ScanDevice(PID, VID)` has its **arguments swapped**; VID/PID = `0x1001/0x8023`.

> **Impact of C1:** every control tick pushes a full frame. This is not a latency bottleneck (asynchronous upload in a worker, with a `join` of the previous one), but it implies **always setting the display image** — decide upfront which face to show (e.g., static eyes) and the target rate (~20–30 Hz).

🕳️ Identified gaps

Foundation (entirely missing)

- **No git.** Before touching firmware/SW: `git init` + **backup of the good** `.hex` (`6.Tests/_Released`) for rollback.
- **Written USB protocol spec** (512/224 framing + 32-byte block). Today it only exists implicitly in the code — a **prerequisite** for the Zephyr port.

- **Reproducible C++ build** (OpenCV 3.4.8 *debug* + `D:\Libraries\...` paths). Only matters if compiling; a Python prototype does not need it.

Setup / prerequisites (blocking)

- **Zadig / driver binding** to the correct VID/PID — the BotDriver `.inf` points to the bootloader. Without this, nothing connects.
- **ServoToolKit calibration**: I2C IDs (2,4,6,8,10,12), zero, PID, limits — **one servo at a time**; requires flashing the toolkit firmware and then restoring the normal one.
- **Power budget**: 6 servos + display on a single USB bus → risk of *brownout*. Limit simultaneous movement.

Front 1 (webcam)

- Detector **+ tracker**; "which face" policy (largest/closest / lock on the owner).
- Loop: **deadzone**, smoothing (EMA), **rate-limit**, beware of **cascading** (servo PID underneath).
- **No-face** behavior (hold/center/park).
- Vertical = digital **crop only** (re-centers on the output).
- Output: **OBS Virtual Camera** (via `pyvirtualcam`); measure end-to-end latency.
- Hot-plug/reconnection; **privacy/park** mode; **0.3MP** camera (limited quality).

Front 2 (PC software)

- **Decision**: Python via the **AHK DLL** (ready-made, C-ABI) ✅ — vs. a custom compiled wrapper. Use **the DLLs from the AHK package** (only they export `AHK_*`).
- **Windows-only** (libusb-win32). Linux/Mac = new port.
- **License** (likely GPL) if distributing.

Front 3 (Zephyr)

- Choose the **Zephyr USB stack** (legacy device vs. experimental `usb_device_next`) — risky; this is where the custom protocol lives.
- GC9A01 **has a Zephyr driver** (`gc9x01x`), but the model is **custom streaming via DMA** → likely bypass of the display subsystem.
- **Flash/recovery**: ST-Link + **DFU** bootloader (0483:5740).
- **Tests** (Ztest/Twister); keep the **2 firmwares coexisting**; non-negotiable criterion = **byte-compatible with the PC SDK**.
- **Honest ROI**: the head is almost a USB↔SPI/I2C bridge; the gain from Zephyr is **maintainability/ecosystem, not function**. The F042 servo (6KB RAM) **stays out**.

Cross-cutting

Acceptance criteria per phase (DoD), risk register, latency metric, and defining the **objective** (personal use vs. distribution).

🗺️ Revised roadmap

Phase	Deliverable	Definition of Done
-1 · Foundation	`git init` + `.hex` backup + **protocol spec (1 page)** + [build env, if compiling]	Versioned repo, `.hex` saved, spec written
0 · Bring-up & setup	Zadig + BotDriver; ServoToolKit (IDs/zero/limits/PID); run `Sample.exe`	"Robot connected!" + `happy.mp4` on the face **and** responds to `SetJointAngles`
1 · Webcam-follow (Python)	camera→face→tracker→**P/PI on j6 (±90, deadzone/EMA/rate-limit)**, undo 180° flip, static eyes on the display, no-face→center; then **virtual cam**	Face centered on the horizontal axis; output usable in Zoom/Teams
2 · Zephyr (head)	spec → `usbd` PoC reproduces the protocol (PC SDK connects) → SPI/DMA display → I2C servos → parity	`Sample.exe` works **identically** against the Zephyr firmware
3 · Servos (optional)	only if the MCU is swapped; otherwise keep the **LL** build	—

🔥 Risk register

Risk	Sev.	When	Mitigation
Driver binding / Zadig	High	Phase 0	Validate the connection before any new code
Brownout with 6 servos on USB	Medium	Phase 0/1	Limit simultaneous movement; measure current; only move j6 in Phase 1
Reimplementing custom USB on Zephyr byte-by-byte	High	Phase 2	Spec first; test against `Sample.exe` at every step
Cascade loop oscillation	Medium	Phase 1	EMA + deadzone + rate-limit; velocity control
Low functional ROI of Zephyr	Medium	Phase 2	Decide whether the objective is learning/maintenance before investing

Recommended sequence: Phase -1 → 0 → 1 (visible result, low risk, zero firmware) before tackling Phase 2.